

1. How Internet Works

The internet is a global network of connected computers that talk to each other using a shared set of rules (protocols).

Steps	What happens
1. You type a URL	e.g. example.com
2. DNS lookup	Your device asks a DNS server to translate that domain into an IP address.
3. Connection	Browser opens a connection to the server at that IP address.
4. Request sent	Browser sends an HTTP(S) request asking for the page.
5. Server responds	The hosting server processes the request and sends back HTML, CSS, JS, or data.
6. Browser renders	Your browser builds the page from what it received and displays it.

2. Client-Server Architecture

A client sends a request and a server sends back a response. The browser on your laptop is a client. The machine running the website's backend code is the server. Every page load, API call, or file download follows this same pattern.

Term	Role
Client	Initiates requests (browser, mobile app, or another server).
Server	Listens for requests, processes them, and sends a response.
Request	What the client sends to the server (URL, HTTP method, headers, and sometimes data).
Response	What the server sends back to the client (status code, headers, and data).

3. Browser

The browser takes a URL, resolves it to a server address, sends an HTTP request, and renders whatever comes back. Rendering means parsing HTML into a page structure, applying CSS for layout and style, and running JavaScript for behavior.

4. HTTP vs HTTPS

HTTP (HyperText Transfer Protocol) is the standard protocol used for communication between a web browser (client) and a web server. Data is transmitted in plain text, making it vulnerable to interception.

HTTPS (HyperText Transfer Protocol Secure) is the secure version of HTTP. It uses TLS (Transport Layer Security) formerly known as SSL (Secure Sockets Layer) to encrypt communication, ensuring that data remains confidential and cannot be modified during transmission.

Feature	HTTP	HTTPS
Full Form	HyperText Transfer Protocol	HyperText Transfer Protocol Secure
Default Port	80	443
Encryption	No encryption	Uses TLS/SSL encryption
Data Security	Data is transmitted in plain text	Data is encrypted before transmission
Privacy	Anyone intercepting the traffic can read the data	Intercepted data is unreadable without the encryption key
Data Integrity	Data can be modified during transmission	Detects any tampering with transmitted data
Authentication	Does not verify the server's identity	Verifies the server using a digital certificate
Browser Indicator	Often displays "Not Secure"	Displays a padlock icon indicating a secure connection
URL Prefix	http://	https://

5. Request–Response Cycle

The Request–Response Cycle is the process through which a client (such as a web browser or mobile application) communicates with a server. Every webpage load, API request, form submission, or button click that requires server interaction follows this cycle. The client sends an HTTP request, the server processes it, and then returns an HTTP response.

Stages

Stage	Description
1. Request Built	The client creates an HTTP request by specifying the HTTP method (e.g., GET, POST), URL, headers, and optionally a request body containing data.
2. Request Sent	The request is transmitted over the network (typically via TCP/IP and HTTP/HTTPS) to the destination server.
3. Server Processes Request	The server receives the request, identifies the appropriate route or endpoint, executes the required application logic, and may interact with databases, files, or external services.
4. Response Built	After processing, the server prepares an HTTP response containing a status code, response headers, and an optional response body (such as HTML, JSON, XML, or files).
5. Response Sent	The server sends the completed response back to the client through the network.
6. Client Handles Response	The client interprets the response. A web browser may render HTML, while a JavaScript application may parse JSON and update the user interface dynamically.

6.. Domain, DNS, and Hosting

Domain: A human-readable name for a website, e.g. example.com. Registered through a domain registrar.

DNS(Domain Name System): The system that translates domain names into IP addresses, so browsers know which server to contact.

Hosting: The server (or cloud service) that stores a website's files and serves them to visitors. When you type a URL, the browser asks a DNS server to resolve the domain to an IP address, then connects to the hosting server at that address.

6. Frontend vs Backend



Feature	Frontend	Backend
Definition	The user-facing part of a web application that runs in the browser.	The server-side part that processes requests, executes business logic, and manages data.
Runs On	Client's web browser	Web server or cloud server
Primary Responsibility	Displays the user interface and handles user interactions	Processes requests, manages data, authentication, and business logic
Interacts With	Users	Frontend, databases, and external services
Languages	HTML, CSS, JavaScript, TypeScript	JavaScript (Node.js), Python, Java, PHP, C#, Go, Ruby, etc.
Frameworks/Libraries	React, Angular, Vue.js,	Express.js, Django, Flask, Spring Boot, Laravel, ASP.NET Core
Database Access	Does not directly access databases	Directly communicates with databases
Output	User interface (HTML, CSS, images, animations)	Data (usually JSON), HTML, files, or API responses
Security	Limited; code is visible to users	Secure; source code and credentials remain on the server

Examples	Login page, navigation bar, buttons, forms	User authentication, payment processing, database operations, API endpoints
-----------------	--	---

7. Full Stack

Full Stack refers to all the layers of a web application, including the frontend, backend, and database.

A Full Stack Developer is someone who can develop and work on the entire application from the user interface that users see to the server-side logic and the database that stores data.

8.REST(Restfull) API

An Application Programming Interface (API) is a set of rules and protocols that enables different software applications to communicate and exchange data with one another. It acts as an intermediary between a client and a server, allowing the client to request specific services or data without needing to understand the server's internal implementation.

In simple terms: An API is a way for two software applications to communicate with each other.

Example:

1. You tell the waiter what food you want.
2. The waiter takes your order to the kitchen.
3. The kitchen prepares the food.
4. The waiter brings the food back to you.

Here, the **waiter is the API** because it carries requests and responses between you and the kitchen.

REST (Representational State Transfer)

Representational State Transfer (REST) is an architectural style for designing web services and APIs. It defines a set of principles that enable communication between clients and servers using the HTTP protocol.

REST organizes data as resources, which are identified by unique URLs. These resources are manipulated using standard HTTP methods such as GET, POST, PUT, PATCH, and DELETE.

REST itself is not an API. Instead, it is a collection of design principles that guide how web APIs should be structured.

Finally ,

RESTful API

A RESTful API is an Application Programming Interface that follows the architectural principles defined by REST.

A RESTful API uses:

- Resource-based URLs (e.g., /users, /products)
- Standard HTTP methods (GET, POST, PUT, PATCH, DELETE)
- Stateless communication, where each request contains all the information required for processing
- Standard HTTP status codes (e.g., 200 OK, 404 Not Found)
- Data formats such as **JSON**, which is the most commonly used format for exchanging data.

Simple Working Flow of a RESTful API

Step 1: Client Sends an HTTP Request

Suppose the React application needs to retrieve a list of all users. The frontend initiates the request using the `fetch()` function:

```
fetch("/users");
```

By default, the `fetch()` function sends an **HTTP GET** request. Consequently, the browser sends the following request to the backend server:

```
GET /users
```

This request asks the server to retrieve the collection of users.

Step 2: Backend Receives and Processes the Request

Upon receiving the request, the backend server examines two key components:

- **HTTP Method:** GET
- **Request Path (Endpoint):** /users

The server then searches its registered API routes for a matching endpoint. In an Express.js application, the corresponding route may be defined as follows:

```
app.get("/users", getUsers);
```

Since both the HTTP method and the endpoint match the incoming request, the server invokes the `getUsers()` handler function to process the request.

Step 3: Business Logic Retrieves Data from the Database

The `getUsers()` function communicates with the database to retrieve the requested user information. A simplified implementation is shown below:

```
function getUsers(req, res) {  
  const users = database.findAll();  
  res.json(users);  
}
```

The function performs the following operations:

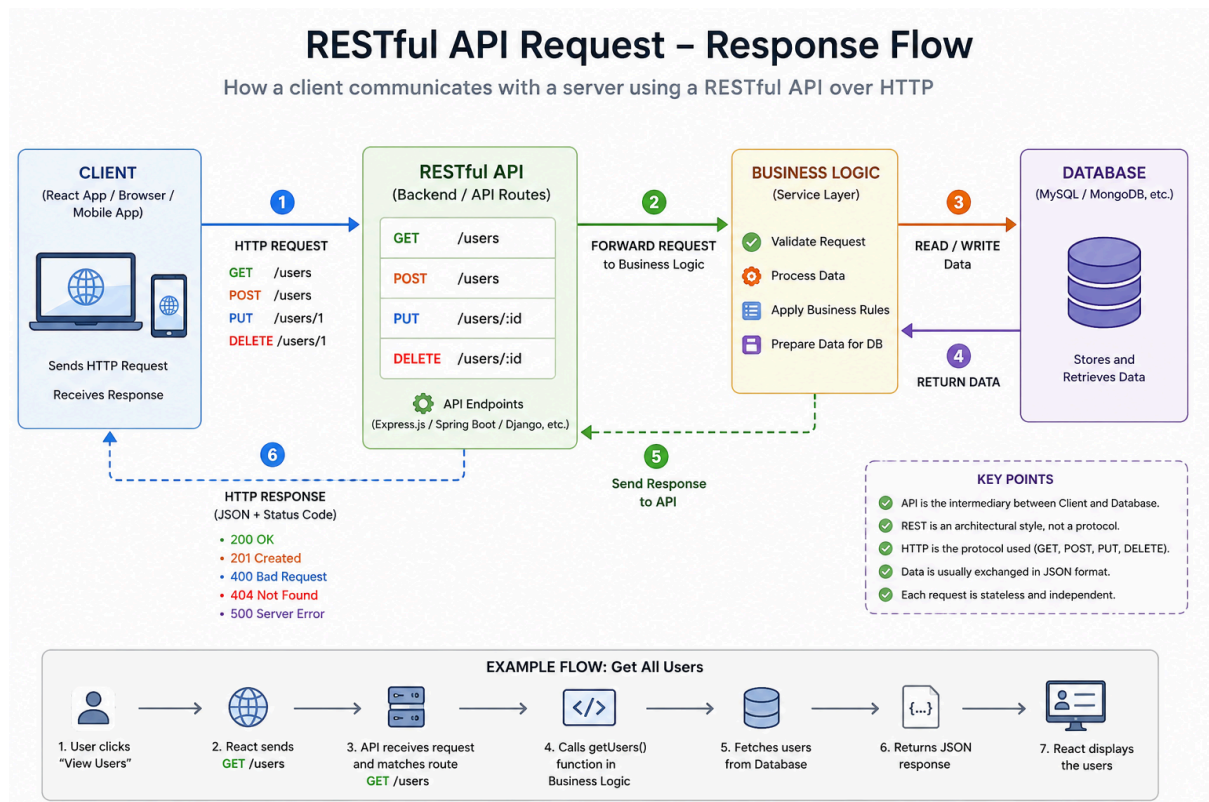
1. Retrieves all user records from the database.
2. Converts the retrieved data into JSON format.
3. Sends the JSON data back to the client as the HTTP response.

Step 4: Server Returns the Response

After successfully retrieving the data, the backend server returns an HTTP response containing the user information, typically in **JSON** format.

```
[  
  {  
    "id": 1,  
    "name": "Alice"  
  }  
]
```

Upon receiving the response, the React application parses the JSON data and updates the user interface, displaying the list of users to the user.



9.React

React is a free and open-source JavaScript library for building user interfaces (UIs), especially for Single Page Applications (SPAs). It was developed by Meta (formerly Facebook) and is used to create fast, interactive, and reusable web applications. Unlike a framework, React focuses only on the view layer (UI) of an application. Other libraries or tools are often used alongside React for routing, state management, and backend communication.

Why React?

Before React, developers manually updated the DOM (Document Object Model) whenever data changed, which became difficult and inefficient for large applications. React simplifies this by automatically updating only the parts of the page that have changed, making applications faster and easier to maintain.

Key Features of React

Feature	Description
Component-Based	Applications are built using reusable components.
Virtual DOM	React updates only the changed parts of the DOM, improving performance.

JSX	Allows developers to write HTML-like syntax inside JavaScript.
One-Way Data Flow	Data flows from parent components to child components through props.
Declarative UI	Developers describe what the UI should look like, and React updates it automatically.
Reusable Components	Components can be reused throughout the application, reducing code duplication.
Fast Rendering	Uses the Virtual DOM to minimize unnecessary updates.

11.Internship Expectation

- Apply my technical knowledge to real-world software projects.
- Gain experience working in a professional development team.
- Learn industry-standard coding practices and software architecture.
- Improve my problem-solving and debugging skills.
- Understand Agile development workflows, code reviews, and collaborative development.
- Enhance my communication and teamwork skills in a professional environment.
- Receive mentorship and constructive feedback to grow as a software developer.
- Contribute meaningfully to projects while expanding my practical experience.